

SERVERLESS SEASONAL STYLE HUB SHOPPING PLATFORM USING AWS

Prepared in the partial fulfillment of the Summer Internship Program on AWS

AT



Under the guidance of

Mrs. Sumana Bethala, APSSDC

Mr. J.Lovababu, APSSDC

Puli Zeneswari, 23BQ1A05I5, VVIT GUNTUR, (Batch -1)

Pothina Pavan Kalyan, 24BQ5A0519, VVIT GUNTUR, (Batch -1)

Palaparthi Ramya, 23BQ1A05G6, VVIT GUNTUR, (Batch -1)

Pulipaka Srinivasa Rao, 24BQ5A0520, VVIT GUNTUR, (Batch -1)

Oturi Lakshmi Haneela, 24BQ5A0518, VVIT GUNTUR, (Batch -1)

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to all those who have contributed to the successful completion of my summer internship project at **Andhra Pradesh Skill Development Corporation (APSSDC)**. This opportunity has been an enriching and transformative experience for me, and I am truly thankful for the support, guidance, and encouragement I have received along the way.

First and foremost, I extend my sincere regards to Mrs Sumana Bethala and Mr J Lova Babu, my supervisors and mentors, for providing me with valuable insights, constant guidance, and unwavering support throughout the duration of the internship. Their expertise and encouragement have been instrumental in shaping the direction of this project.

I would like to thank the entire team at **Andhra Pradesh Skill Development Corporation (APSSDC)** for fostering a collaborative and innovative environment. The camaraderie, knowledge sharing, and feedback I received from my colleagues significantly contributed to the development and success of this project.

In conclusion, I am honoured to have been a part of this internship program, and I look forward to leveraging the skills and knowledge gained to contribute positively to future endeavours.

Thank you.

Sincerely,

Puli Zeneswari, 23BQ1A05I5, zeneswaripuli@gmail.com.

Pothina Pavan Kalyan, 24BQ5A0519, kalyanpothinapavan@gmail.com.

Palaparthi Ramya, 23BQ1A05G6, pramya1409@gmail.com.

Pulipaka Srinivasa Rao, 24BQ5A0520, srinivaspulipaka309@gmail.com.

Oturi Lakshmi Haneela, 24BQ5A0518, lakshmihaneelaoturi@gmail.com.

ABSTRACT

This project presents the design and implementation of Seasonal Styles Hub, a serverless e-commerce web application built entirely on Amazon Web Services (AWS). The platform allows global users to browse a seasonal clothing catalogue, create an account, sign in securely, and manage a shopping cart, all without the project team having to provision or manage a single server.

The frontend of the application is a static website hosted on Amazon S3 (bucket seasonal-styles-hub-v2), delivering the storefront pages — Home, About Us, Contact, Cart, and Sign In — directly over HTTPS to users anywhere in the world. User authentication and identity management are handled by an Amazon Cognito User Pool, which verifies user credentials before granting access to protected shopping-cart operations.

Shopping-cart functionality is exposed through a REST API built with Amazon API Gateway (SeasonalShoppingAPI), which routes /cart and /products requests to an AWS Lambda function, ManageShoppingCart, implemented in Node.js/Python. This function reads from and writes to an Amazon DynamoDB table, ShoppingCarts, which uses cartId as its primary key and offers low-latency, highly available NoSQL storage for cart data. AWS IAM roles enforce least-privilege access between Lambda, DynamoDB, and the API layer, while the entire stack is provisioned and version-controlled using AWS CloudFormation, enabling repeatable, infrastructure-as-code deployments.

The resulting architecture demonstrates how a fully serverless, event-driven design can deliver a scalable, secure, and cost-efficient e-commerce experience with minimal operational overhead, while remaining straightforward to extend with future capabilities such as payment processing, content delivery acceleration, and personalized recommendations.

TABLE OF CONTENTS

S.No	Content	Pg.No
1.	INTRODUCTION	5
2.	METHODOLOGY	7
3.	TECHNOLOGY STACK	10
4.	SYSTEM DESIGN / ARCHITECTURE	13
5.	IMPLEMENTATION	25
6.	RESULTS.....	30
7.	CONCLUSION	33

1. INTRODUCTION

In the digital era, where agility, automation, and real-time customer experience have become central to retail success, online shopping platforms are expected to be fast, secure, and available at any scale without downtime. This project, titled Seasonal Styles Hub, harnesses the power of serverless computing on Amazon Web Services (AWS) to deliver a robust, scalable, and fully managed e-commerce storefront for seasonal clothing — Spring Wear, Summer Outfits, Autumn Fashion, and Winter Special collections.

Modern e-commerce businesses demand uninterrupted storefront availability, secure account management, and a shopping-cart experience that responds instantly regardless of how many customers are browsing at once. Traditional server-based deployments often fall short in this regard, requiring constant capacity planning, patching, and scaling effort. Seasonal Styles Hub meets these challenges by adopting a fully serverless architecture, where AWS managed services work together to host the storefront, authenticate shoppers, and process cart operations automatically, all while maintaining high availability and operational efficiency.

The system is designed with simplicity, security, and automation at its core. Amazon S3 hosts the static storefront — the Home, About Us, Contact, Cart, and Sign In pages — and serves them directly to global users over HTTPS. Amazon Cognito manages user identity, authenticating shoppers such as the sample registered user shown in the architecture (kalyanpothinapavan) before they can access cart functionality. Amazon API Gateway exposes a REST API, `SeasonalShoppingAPI`, that fronts an AWS Lambda function, `ManageShoppingCart`, which processes `/cart` and `/products` requests and reads and writes cart records in Amazon DynamoDB.

What sets this project apart is its strategic use of AWS's serverless building blocks, eliminating the need for server provisioning, manual scaling, or ongoing infrastructure maintenance. Each component is invoked only when needed, enabling optimal resource usage and cost efficiency. AWS IAM guarantees that only the permissions each service genuinely needs are granted — Lambda, for instance, is authorized to access DynamoDB but nothing beyond its remit — while AWS CloudFormation captures the entire stack as version-controlled infrastructure-as-code, so the whole environment can be reproducibly deployed or torn down.

This report provides a comprehensive overview of the system's design, architecture, and implementation, highlighting the decision-making process, technical integrations, and benefits realized through a serverless approach. The architecture promotes modularity and extensibility, enabling straightforward integration with future enhancements such as a payment gateway, a content delivery network in front of the storefront, order-history tracking, or personalized product recommendations.

By delivering a real-time, secure, and cloud-native shopping experience, this project not only addresses the immediate needs of a seasonal fashion retailer but also showcases the broader potential of serverless technologies in solving real-world e-commerce challenges across industries.

2. METHODOLOGY

The development of Seasonal Styles Hub followed a structured and iterative methodology to ensure the successful fulfilment of the project's objectives. The methodology was divided into key phases, including requirements gathering, system design, implementation, testing, deployment, and refinement. Each phase was designed to align with best practices in cloud-native, serverless development. The following subsections outline each phase in detail.

2.1. Requirements Gathering

The project began with a thorough analysis of the functional and technical requirements of a seasonal fashion storefront. The core objectives identified were: present a browsable catalogue of seasonal collections, allow customers to create an account and sign in securely, let signed-in customers add and manage items in a shopping cart, and ensure the entire platform runs without provisioning or managing traditional servers. The system was also required to scale automatically to handle unpredictable traffic spikes, such as seasonal sales events, without manual intervention.

2.2. Design

Based on the identified requirements, a cloud-native, event-driven architecture was designed using AWS managed services. The workflow was defined to include Amazon S3 as the presentation layer, Amazon Cognito for identity and access, Amazon API Gateway and AWS Lambda for the cart and product processing logic, and Amazon DynamoDB for cart data storage. IAM roles were planned to ensure secure, least-privilege interactions between services, and AWS CloudFormation was chosen as the deployment mechanism so the whole stack could be defined, versioned, and reproduced as infrastructure-as-code. Special attention was given to decoupling components so that the frontend, identity layer, API layer, and data layer could evolve and scale independently.

2.3. Implementation

The implementation phase focused on translating the design into a fully functional serverless application. The storefront pages were built as static HTML/CSS/JavaScript and uploaded to the seasonal-styles-hub-v2 S3 bucket, which was configured for static website hosting. A

Cognito User Pool was created to register and authenticate shoppers. The SeasonalShoppingAPI REST API was configured in API Gateway with resources for /cart and /products, both integrated with the ManageShoppingCart Lambda function. The Lambda function itself was developed to read incoming API requests, validate the payload, and perform the corresponding create, read, update, or delete operation against the ShoppingCarts DynamoDB table.

2.4. Database Management

A NoSQL approach was adopted for efficient, low-latency cart storage using Amazon DynamoDB. The ShoppingCarts table schema uses cartId as its primary key, allowing the Lambda function to retrieve, update, and delete a specific shopper's cart in a single, predictable operation. This design supports fast reads and writes at any scale and integrates seamlessly with Lambda through the AWS SDK. Data integrity was maintained through payload validation performed inside the Lambda function prior to any write operation.

2.5. Testing

Multiple levels of testing were carried out to validate functionality, error handling, and system performance. Unit-style tests exercised the Lambda function's request parsing, validation, and cart-mutation logic in isolation. Integration testing confirmed that API Gateway, Lambda, DynamoDB, and Cognito worked together correctly end to end — from a user signing in, through adding an item to the cart, to the updated cart being persisted and retrievable. Functional testing simulated typical shopper journeys across the Home, Cart, and Sign In pages to confirm the storefront behaved correctly under normal use.

.

2.6. Deployment

The system was deployed to AWS using AWS CloudFormation, which codifies the S3 bucket, Cognito User Pool, API Gateway API, Lambda function, DynamoDB table, and the associated

IAM roles as a single, version-controlled template. This infrastructure-as-code approach allows the entire environment to be created, updated, or removed consistently and repeatably, removing the risk of manual configuration drift between environments. IAM roles were carefully scoped to grant only the permissions each service required, and the static website endpoint was verified to confirm the storefront was reachable over HTTPS.

2.7. Iterative Refinement

Throughout the development cycle, iterative feedback loops were established to continuously refine the system. Adjustments were made to the API resource structure, the Lambda function's error handling, and the Cognito sign-in flow based on test results. The architecture was reviewed periodically to ensure adherence to serverless best practices, and the CloudFormation template was updated to reflect each refinement, keeping the deployed environment and the infrastructure-as-code definition in sync.

2.8. High-Level Data Flow

At a high level, the methodology can be summarised by the path a single shopper interaction takes through the system. A shopper's browser first requests the storefront's static assets from the S3 website endpoint. If the shopper signs in, their credentials are exchanged with the Cognito User Pool for an authentication token. Every subsequent request to a protected resource — such as adding an item to the cart — is sent to the SeasonalShoppingAPI in API Gateway together with that token. API Gateway validates the token via its Cognito authorizer, then invokes the ManageShoppingCart Lambda function, which reads or writes the corresponding item in the ShoppingCarts DynamoDB table and returns a JSON response back through API Gateway to the browser.

Designing the methodology around this single, well-understood data flow made it straightforward to reason about failure modes at each stage — for example, what should happen if a token has expired, or if a requested cartId does not yet exist — and to assign the corresponding IAM permissions and error-handling logic to the correct component.

The methodology adopted for the development of Seasonal Styles Hub ensured a cloud-native, scalable, and efficient solution. By following an event-driven and modular approach, the project successfully demonstrates the practical application of serverless technologies in solving real-world e-commerce challenges.

3. TECHNOLOGY STACK

Seasonal Styles Hub was built using a modern, cloud-native technology stack that supports event-driven architecture, secure identity management, and real-time API-driven processing. The following technologies and services were used throughout the implementation.

3.1. Cloud Platform

- Amazon Web Services (AWS): The entire project is hosted on AWS, leveraging its serverless and managed services for hosting, identity, computation, and data storage

3.2. AWS Services

- Amazon S3: Hosts the static storefront (seasonal-styles-hub-v2) and serves Home, About Us, Contact, Cart, and Sign In pages directly over HTTPS.
- Amazon Cognito User Pool: Registers and authenticates shoppers before they can access cart operations.
- Amazon API Gateway: Exposes the SeasonalShoppingAPI REST API, routing /cart and /products requests to the backend.
- AWS Lambda: Executes the ManageShoppingCart function, containing the core business logic for cart and product processing.
- Amazon DynamoDB: NoSQL database storing shopping-cart records in the ShoppingCarts table, keyed by cartId.
- AWS IAM (Identity and Access Management): Provides secure, least-privilege access control between Lambda, DynamoDB, and API Gateway.
- AWS CloudFormation: Defines and deploys the entire stack as version-controlled infrastructure-as-code.

3.3. Programming & Scripting

- Node.js / Python (Lambda Runtime): Used for writing the ManageShoppingCart backend logic.

- HTML, CSS, and JavaScript: Used to build the static storefront pages hosted on Amazon S3.
- AWS SDK: Used within Lambda to interact with Amazon DynamoDB and other AWS services.

3.4. Data Format

JSON (JavaScript Object Notation) is used as the request and response payload format between the storefront, API Gateway, and the ManageShoppingCart Lambda function, as well as the internal representation of items within a DynamoDB cart record.

3.5. Security & Access

IAM roles and policies are configured to allow only the necessary permissions for each service, following the principle of least privilege. Cognito manages user credentials and issues tokens that authorize access to the protected /cart resource in API Gateway.

3.6. Infrastructure as Code

AWS CloudFormation templates describe every resource in the stack — the S3 bucket and its website configuration, the Cognito User Pool, the API Gateway API and its resources, the Lambda function, the DynamoDB table, and the supporting IAM roles — allowing the entire environment to be deployed, updated, or destroyed consistently and repeatably.

3.7. Why a Serverless Stack

Choosing a serverless stack for Seasonal Styles Hub was a deliberate decision driven by the nature of e-commerce traffic, which is often unpredictable and seasonal by definition. A traditional, server-based deployment would require the project team to provision fixed compute capacity for the storefront and API layer, estimate peak load in advance, patch and secure the underlying operating systems, and pay for idle capacity outside of peak shopping periods. The

table below summarises the practical differences observed between a traditional server-based deployment and the serverless approach adopted for this project.

Aspect	Traditional (Server-Based)	Serverless (This Project)
Capacity planning	Manual sizing of EC2 instances ahead of demand	Automatic — AWS scales each service on demand
Idle cost	Paid for around the clock, even off-peak	Pay-per-request; near-zero cost when idle
Patching & OS maintenance	Required for every server	None — fully managed by AWS
High availability	Must be designed and configured explicitly	Built into each managed service by default
Deployment	Manual or scripted server configuration	Declarative CloudFormation template

This comparison reinforced the decision to build every layer of Seasonal Styles Hub — hosting, identity, API, compute, and storage — on managed, serverless AWS services rather than on self-managed virtual machines.

4. SYSTEM DESIGN / ARCHITECTURE

Seasonal Styles Hub is designed as an event-driven, cloud-native solution that automates storefront delivery, user authentication, and shopping-cart processing. The architecture adopts a fully serverless model, ensuring scalability, minimal infrastructure maintenance, and cost-efficiency. The system follows a multi-tier design, comprising the frontend layer, security and identity layer, backend and integration layer, and database layer, all powered by managed AWS services within the US-EAST-1 region.

This section outlines the core components and their interactions in the architecture, as illustrated in Figure 1.

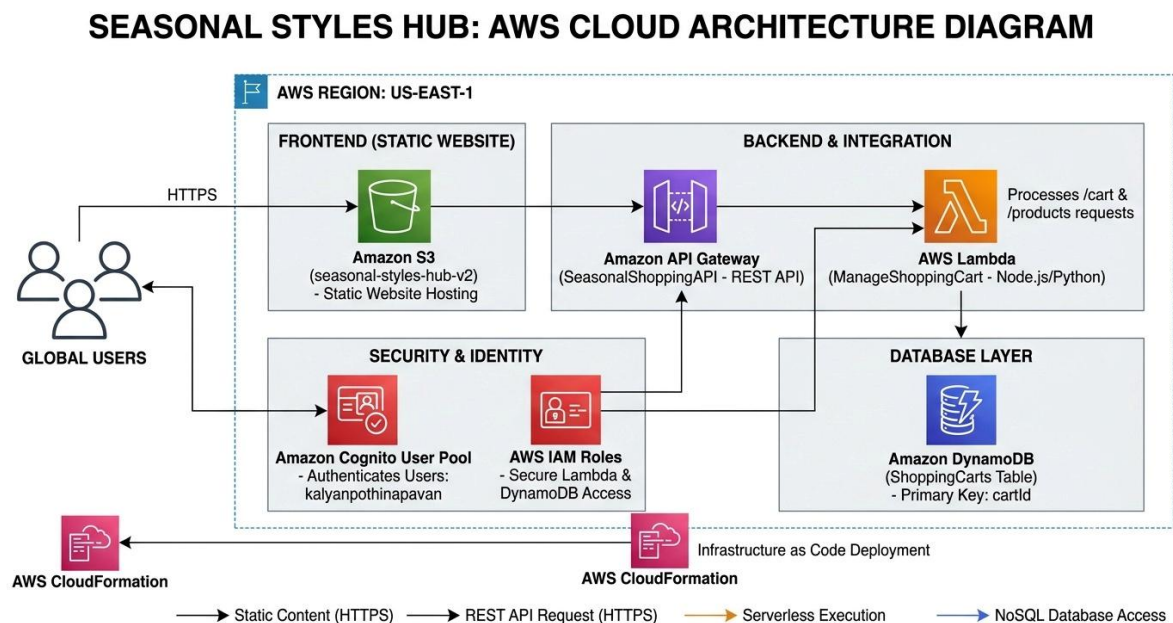


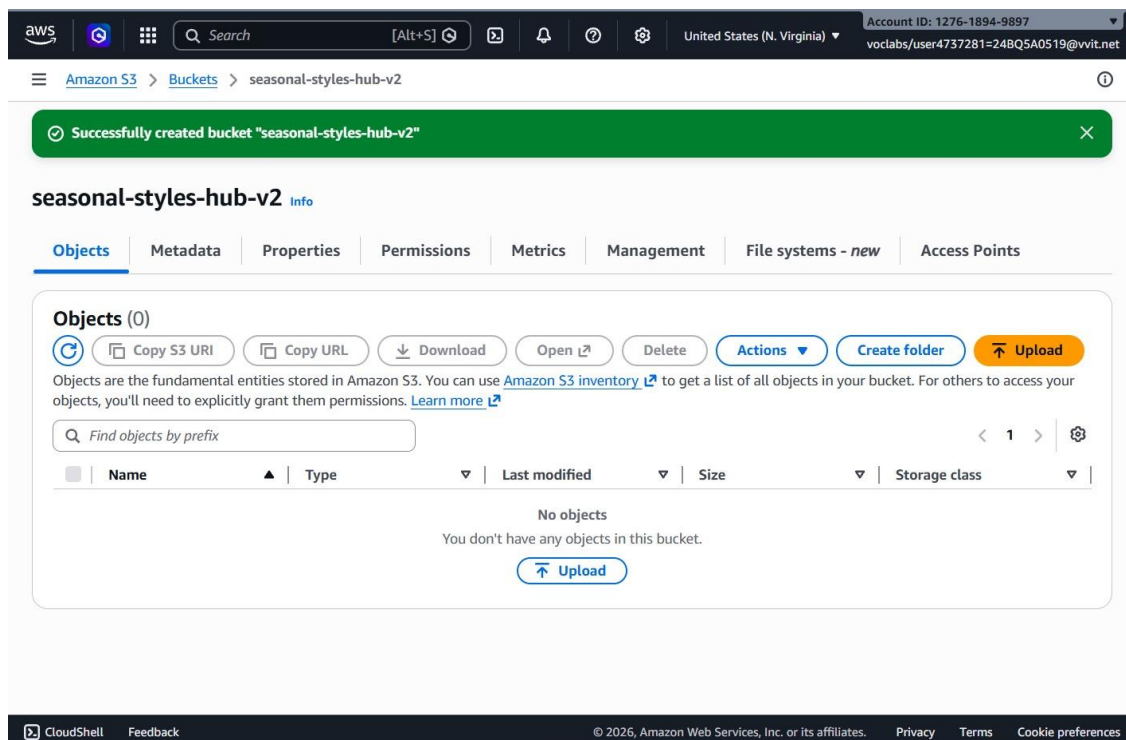
Figure 1. Seasonal Styles Hub — AWS Cloud Architecture Diagram

4.1. Frontend — Amazon S3 (Static Website Hosting)

The storefront is a static website hosted in the seasonal-styles-hub-v2 Amazon S3 bucket. S3's static website hosting feature serves the Home, About Us, Contact, Cart, and Sign In pages directly to browsers over HTTPS, with no web server to patch or scale. Global users reach the storefront through a simple HTTPS request, and S3 automatically handles the resulting read traffic regardless of how many shoppers browse at once.

The homepage presents an All Seasons Collection with four seasonal categories — Spring Wear, Summer Outfits, Autumn Fashion, and Winter Special — alongside navigation to the About Us, Contact, Cart, and Sign In pages.

- Highly durable, low-cost object storage repurposed as a static web host
- Serves HTML, CSS, JavaScript, and image assets directly over HTTPS



aws

Search [Alt+S]

Ask Amazon Q

United States (N. Virginia)

Account ID: 1276-1894-9897
voclabs/user4737281=24BQ5A0519@vvit.net

Amazon S3

Buckets

seasonal-styles-hub-v2

Upload

Upload

Info

Add the files and folders you want to upload to S3. To upload a file larger than 160GB, use the AWS CLI, AWS SDKs or Amazon S3 REST API. [Learn more](#)

Drag and drop files and folders you want to upload here, or choose **Add files** or **Add folder**.

Files and folders (5 total, 5.7 KB)

Remove

Add files

Add folder

All files and folders in this table will be uploaded.

Find by name

< 1 >

☐	Name	Folder	Type	Size
☐	about.html	-	text/html	961.0 B
☐	contact.html	-	text/html	807.0 B
☐	index.html	-	text/html	1.2 KB
☐	script.js	-	text/javascript	1.2 KB
☐	style.css	-	text/css	1.6 KB

Destination

Info

Destination

<s3://seasonal-styles-hub-v2>

CloudShell

Feedback

© 2026, Amazon Web Services, Inc. or its affiliates. [Privacy](#) [Terms](#) [Cookie preferences](#)

Amazon S3

Upload succeeded

For more information, see the Files and folders table.

Summary

Destination

<s3://seasonal-styles-hub-v2>

Succeeded

5 files, 5.7 KB (100.00%)

Failed

0 files, 0 B (0%)

Files and folders

Configuration

Files and folders (5 total, 5.7 KB)

Find by name

< 1 >

Name	Folder	Type	Size	Status	Error
about.html	-	text/html	961.0 B	Succeeded	-
contact.html	-	text/html	807.0 B	Succeeded	-
index.html	-	text/html	1.2 KB	Succeeded	-
script.js	-	text/javascript	1.2 KB	Succeeded	-
style.css	-	text/css	1.6 KB	Succeeded	-

CloudShell

Feedback

© 2026, Amazon Web Services, Inc. or its affiliates. [Privacy](#) [Terms](#) [Cookie preferences](#)

4.2. Security & Identity — Amazon Cognito User Pool

Amazon Cognito provides the identity layer for Seasonal Styles Hub. The Cognito User Pool registers new shoppers, verifies their credentials at sign-in, and issues tokens that the storefront presents when calling protected API resources. In the deployed environment, the User Pool authenticates registered shoppers such as kalyanpothinapavan before they are permitted to reach cart-related functionality.

- Self-service sign-up and sign-in for shoppers
- Issues JSON Web Tokens (JWTs) used to authorize API Gateway requests
- Removes the need to build or operate a custom authentication service

4.3. Security & Identity — AWS IAM Roles

AWS IAM roles secure the interactions between the backend services. A dedicated execution role is attached to the ManageShoppingCart Lambda function, granting it exactly the permissions it needs to read and write the ShoppingCarts DynamoDB table and to write logs — nothing more. This least-privilege configuration limits the blast radius of any single component and follows AWS security best practices.

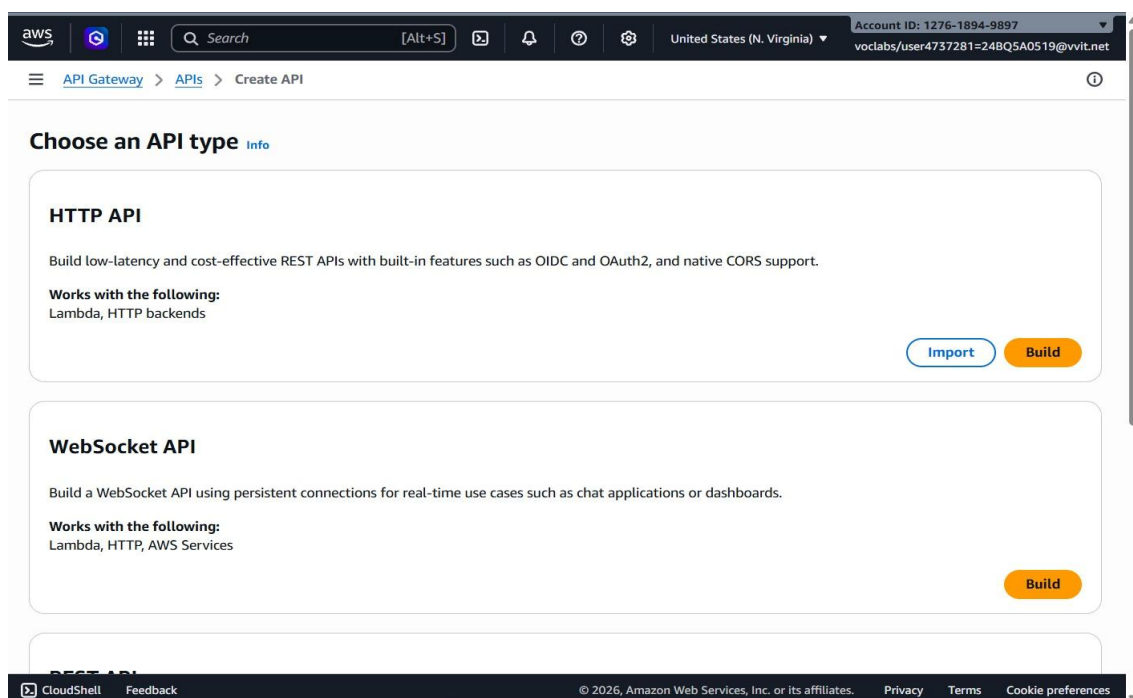
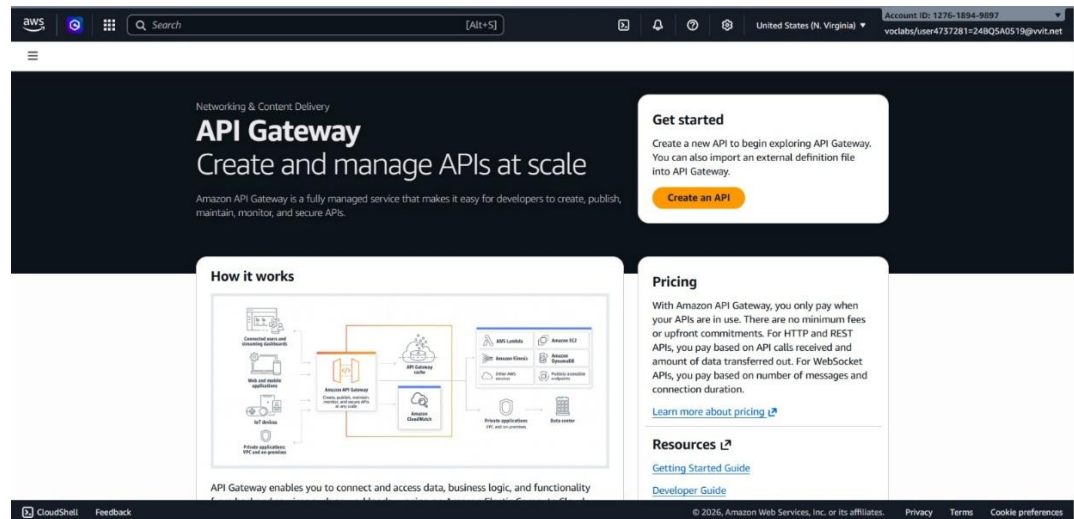
- Scoped execution role for the Lambda function
- Grants DynamoDB read/write access limited to the ShoppingCarts table
- Enforces least-privilege access between all backend services

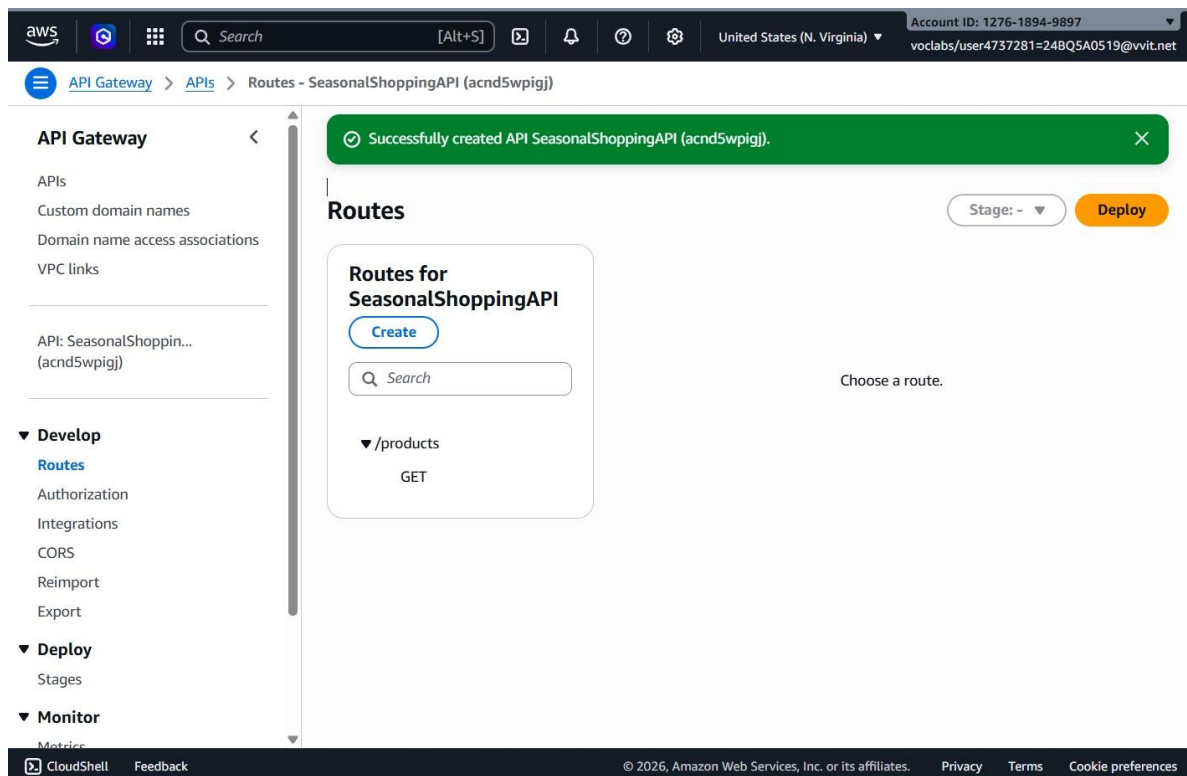
4.4. Backend & Integration — Amazon API Gateway

Amazon API Gateway exposes the SeasonalShoppingAPI, a REST API that acts as the single entry point for all cart and product operations initiated from the storefront. API Gateway validates incoming HTTPS requests, applies the Cognito authorizer to protected routes, and forwards each request to the ManageShoppingCart Lambda function for processing before returning the response to the client.

- Managed REST API endpoint (SeasonalShoppingAPI) fronting the backend logic

- Integrates with Cognito to authorize requests to protected resources
- Decouples the storefront from the implementation details of the backend
-





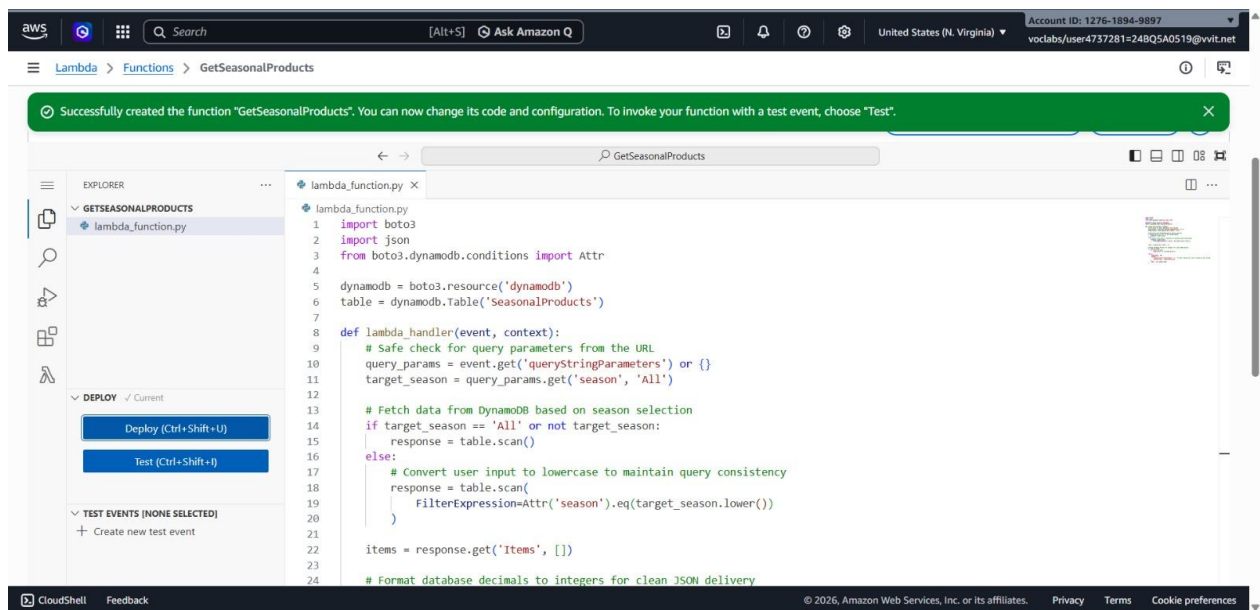
4.5. Backend & Integration — AWS Lambda

The ManageShoppingCart AWS Lambda function, implemented in Node.js/Python, contains the core business logic of the platform. It processes `/cart` and `/products` requests forwarded by API Gateway — retrieving product information, and creating, reading, updating, or deleting items in a shopper's cart — before persisting the result in DynamoDB and returning a response.

- Processes `/cart` and `/products` requests routed from API Gateway
- Stateless, event-driven execution — billed only for actual usage
- Reads from and writes to the ShoppingCarts table in DynamoDB

The screenshot shows the AWS Lambda console interface. At the top, there's a navigation bar with the AWS logo, search bar, and account information. The left sidebar contains navigation links like Dashboard, Applications, Functions, Function resources, MicroVMs, and Network connectors. The main area displays a banner for a webinar, followed by the 'Functions' section header and a table listing five functions.

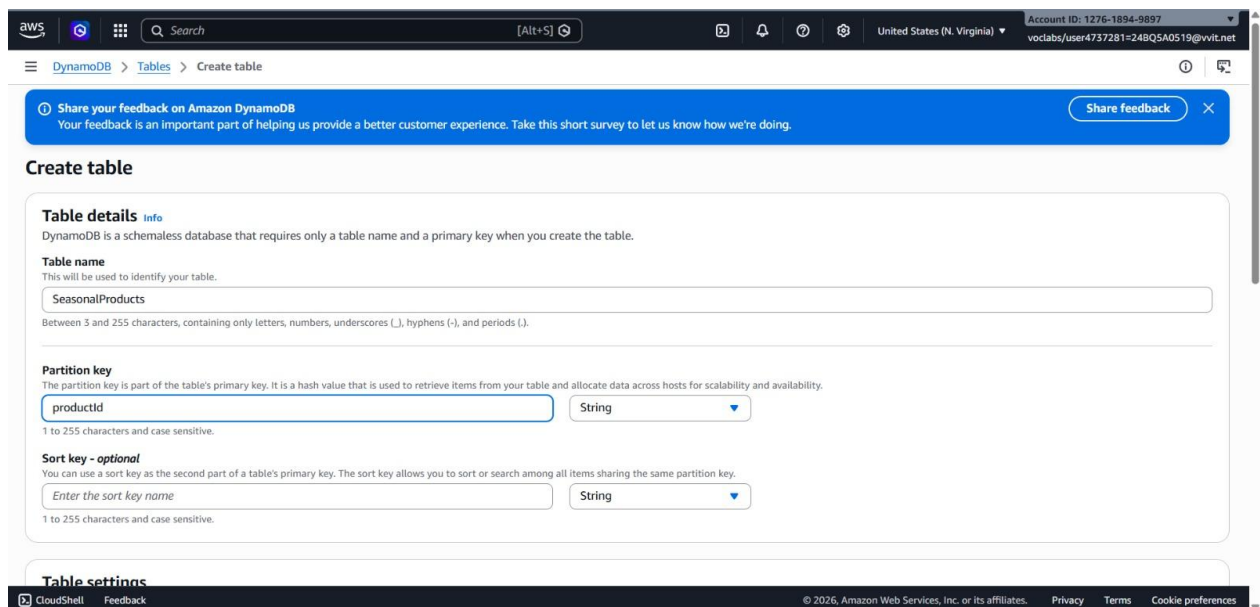
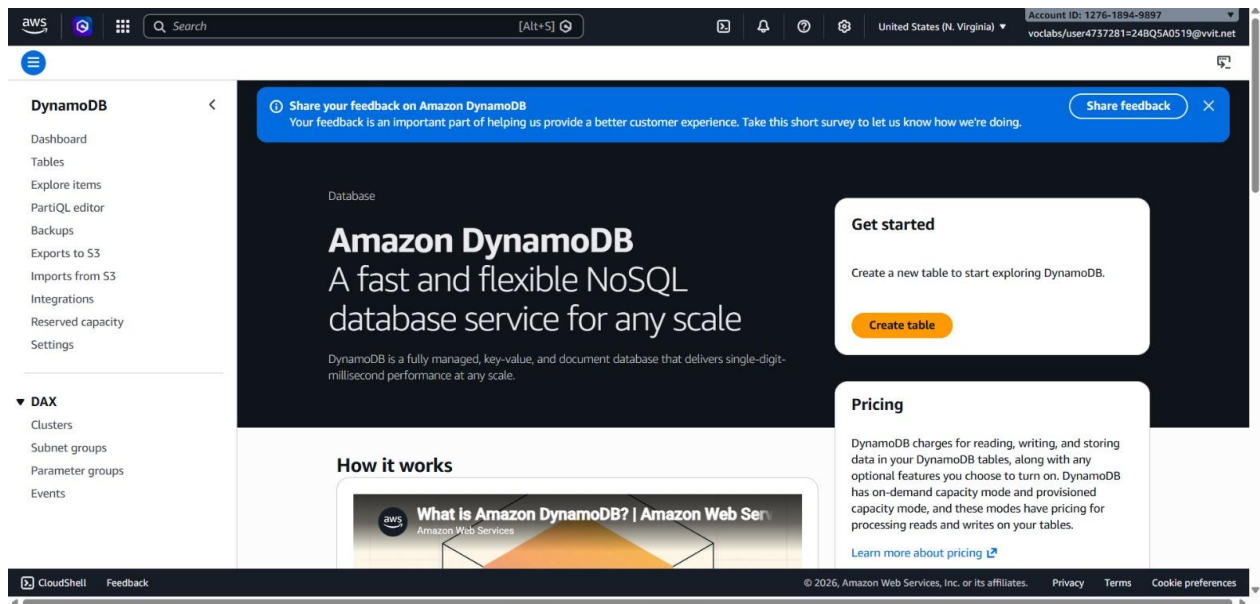
	Function name	Description	Package type	Runtime	Type	Last modified
☐	RoleCreationFunction	Create SLR if absent	Zip	Python 3.10	Standard	2 months ago
☐	RedshiftEventSubscription	Create Redshift event subscription to SNS Topic.	Zip	Python 3.10	Standard	2 months ago
☐	RedshiftOverwatch	Deletes Redshift Cluster if the count is more than 2.	Zip	Python 3.10	Standard	2 months ago
☐	ModLabRole	updates LabRole to allow it to assume itself	Zip	Python 3.10	Standard	2 months ago
☐	MainMonitoringFunction	-	Zip	Python 3.10	Standard	2 months ago



4.6. Database Layer — Amazon DynamoDB

Amazon DynamoDB provides the persistence layer for shopping-cart data. The ShoppingCarts table uses cartId as its primary key, enabling the Lambda function to fetch, update, or remove a specific shopper's cart with single-digit-millisecond latency. As a fully managed NoSQL database, DynamoDB scales automatically with demand and requires no capacity planning or database administration.

- ShoppingCarts table with cartId as the primary key
- Single-digit-millisecond read and write latency at any scale
- Fully managed — no servers, patching, or manual scaling



aws [Search] [Alt+S] United States (N. Virginia) Account ID: 1276-1894-9897 voclabs/user4737281=24BQ5A0519@vvt.net

DynamoDB > Tables > SeasonalProducts

DynamoDB

- Dashboard
- Tables
- Explore items
- PartiQL editor
- Backups
- Exports to S3
- Imports from S3
- Integrations
- Reserved capacity
- Settings

▼ DAX

- Clusters
- Subnet groups
- Parameter groups
- Events

Tables (1)

Filter by tag: Any tag key

Filter by tag value: Any tag value

Find tables

SeasonalProducts

SeasonalProducts Last updated: June 29, 2026, 17:50 (UTC+5:30)

Settings Indexes Monitor Global tables Backups Exports and streams Permissions

Protect your DynamoDB table from accidental writes and deletes

When you turn on point-in-time recovery (PITR), DynamoDB backs up your table data automatically so that you can restore to any given second in the preceding 1 to 35 days. Additional charges apply. [Learn more](#)

Edit PITR

General information

Partition key: productid (String)

Sort key: -

Capacity mode: On-demand

Table status: Active

Alarms: No active alarms

Point-in-time recovery (PITR): Off

Item count: 0

Average item size: 0 bytes

Resource-based policy: Not active

Get live item count

Amazon Resource Name (ARN)

<https://us-east-1.console.aws.amazon.com/dynamodbv2/home?region=us-east-1#item-explorer?maximize=true&table=SeasonalProducts>

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

aws [Search] [Alt+S] Ask Amazon Q United States (N. Virginia) Account ID: 1276-1894-9897 voclabs/user4737281=24BQ5A0519@vvt.net

DynamoDB > Explore items > SeasonalProducts

DynamoDB

- Dashboard
- Tables
- Explore items
- PartiQL editor
- Backups
- Exports to S3
- Imports from S3
- Integrations
- Reserved capacity
- Settings

▼ DAX

- Clusters
- Subnet groups
- Parameter groups
- Events

The item has been saved successfully.

Notifications 0 0 1 1 0

Tables (1)

Filter by tag: Any tag key

Filter by tag value: Any tag value

Find tables

SeasonalProducts

SeasonalProducts Autopreview View table details

▼ Scan or query items

Scan Query

Select a table or index: Table - SeasonalProducts

Select attribute projection: All attributes

Filters - optional

Run Reset

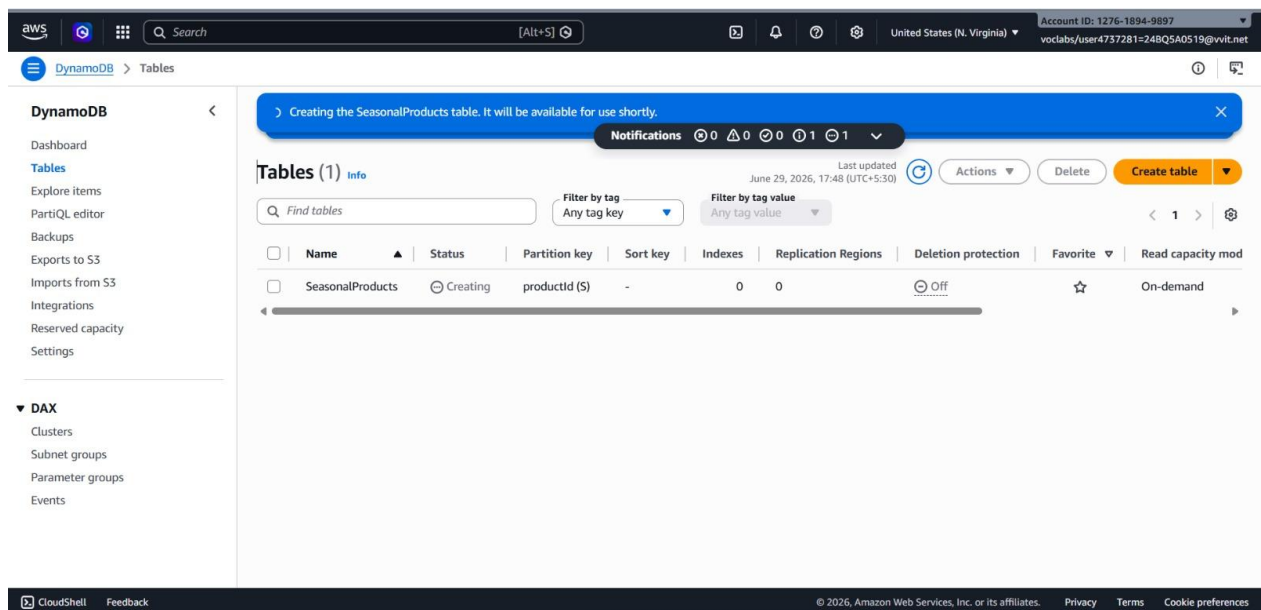
Completed · Items returned: 1 · Items scanned: 1 · Efficiency: 100% · RCUs consumed: 2

Table: SeasonalProducts - Items returned (1)

Actions Create item

CloudShell Feedback

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences



4.7. Infrastructure as Code — AWS CloudFormation

The entire Seasonal Styles Hub stack — the S3 bucket and website configuration, the Cognito User Pool, the API Gateway API and its resources, the ManageShoppingCart Lambda function, the ShoppingCarts DynamoDB table, and the supporting IAM roles — is defined in an AWS CloudFormation template. This infrastructure-as-code approach makes the deployment repeatable and auditable: the same template can recreate an identical environment, and every change to the infrastructure is captured in version control rather than made manually through the console.

- Declarative definition of every AWS resource in the stack
- Repeatable, consistent deployment across environments
- Reduces configuration drift and manual console changes

4.8. End-to-End Data Flow

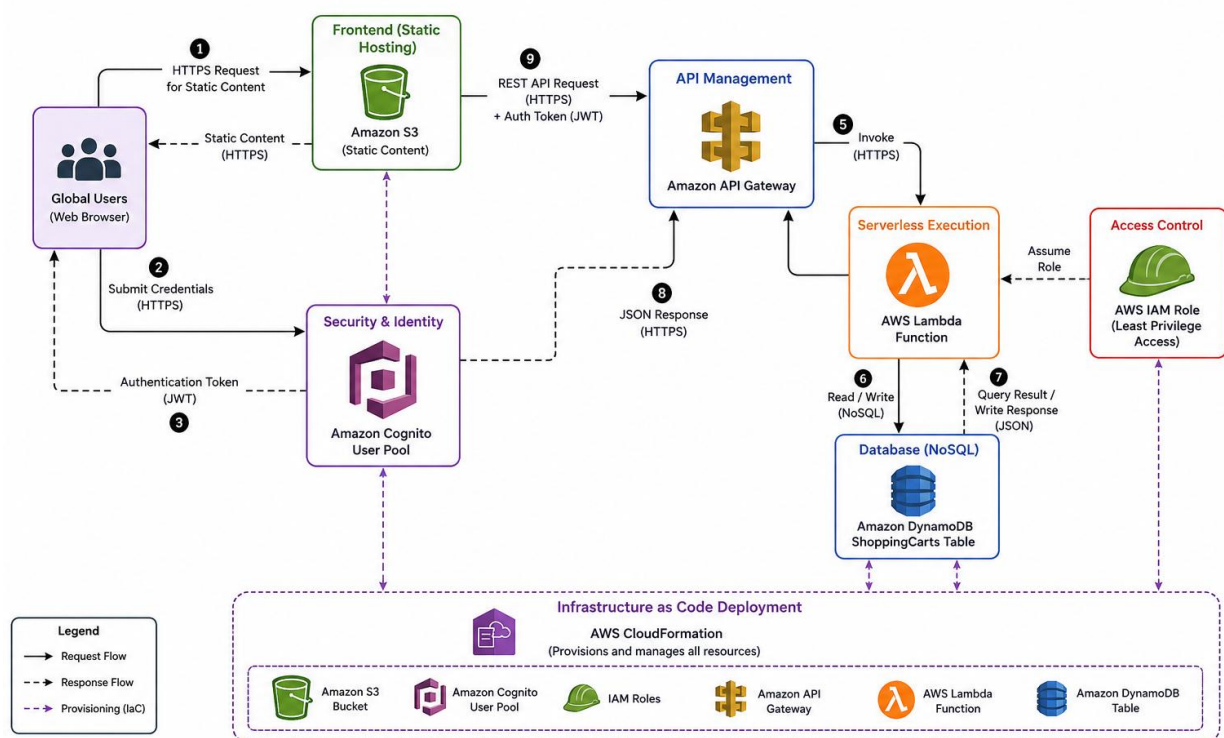
The diagram in Figure 1 can be read as a left-to-right sequence of events. A global user's browser first issues an HTTPS request for static content, which Amazon S3 serves directly (the black "Static Content (HTTPS)" path). When the shopper needs to sign in, the storefront sends their credentials to the Amazon Cognito User Pool, which validates them and returns an

authentication token — this is the interaction shown flowing between Global Users and the Security & Identity block.

Once authenticated, any REST API Request (HTTPS) made by the storefront — for example, adding a product to the cart — travels to Amazon API Gateway, which is shown accepting requests from both the frontend and, indirectly, the identity layer. API Gateway then triggers the AWS Lambda function shown in orange under Serverless Execution, using the AWS IAM Role to assume only the permissions it has been granted. The Lambda function completes the request by reading or writing the Amazon DynamoDB ShoppingCarts table over the NoSQL Database Access path shown in blue, and returns a JSON response that travels back up through API Gateway to the shopper's browser.

Underpinning the entire diagram, AWS CloudFormation (shown at the bottom as an Infrastructure as Code Deployment) is responsible for creating every resource inside the dashed AWS Region boundary — the S3 bucket, the Cognito User Pool, the IAM roles, API Gateway, the Lambda function, and the DynamoDB table — as a single, coordinated deployment.

Serverless E-Commerce Architecture Flow



4.9. Scalability and Fault Tolerance

Each tier of the architecture scales independently and automatically. Amazon S3 absorbs any increase in storefront traffic without configuration changes. Amazon API Gateway and AWS Lambda scale horizontally by creating additional concurrent executions as request volume grows, so a surge of shoppers during a seasonal sale is met with additional compute capacity rather than dropped requests. Amazon DynamoDB scales its throughput to match the read and write demand placed on the ShoppingCarts table. Because none of these services depends on a single, fixed server, the failure of any one underlying host has no visible impact on shoppers — AWS transparently routes traffic to healthy infrastructure behind each managed service.

4.10. Cost Efficiency

The serverless design also has a direct cost benefit. Amazon S3 charges only for the storage consumed and the requests served, AWS Lambda charges only for the compute time actually used while processing a request, Amazon API Gateway charges per API call, and Amazon DynamoDB (in on-demand mode) charges per read and write request. During periods of low shopper traffic — for example, overnight — the platform incurs minimal cost, since idle capacity is never provisioned or paid for, in contrast with a fleet of always-on servers

5. IMPLEMENTATION

The implementation of Seasonal Styles Hub translates the architectural design into a fully functional cloud-based storefront. This section outlines how each AWS service and component was configured and developed to work together in an automated, secure workflow. The implementation focused on delivering a fast static storefront, secure shopper authentication, and reliable cart processing — all without managing any physical or virtual servers.

5.1. Static Storefront Deployment to Amazon S3

The storefront's HTML, CSS, JavaScript, and image assets were uploaded to the seasonal-styles-hub-v2 S3 bucket, and static website hosting was enabled on the bucket so that it could serve pages directly over HTTP/HTTPS. The Home page presents the All Seasons Collection with links into the Spring Wear, Summer Outfits, Autumn Fashion, and Winter Special categories, alongside top-level navigation to About Us, Contact, Cart, and Sign In.

- Bucket configured for static website hosting with an index document
- Public read access scoped to website assets only
- Storefront reachable at the S3 website endpoint over HTTPS

5.2. Shopper Authentication with Amazon Cognito

An Amazon Cognito User Pool was provisioned to handle shopper registration and sign-in. The Sign In page on the storefront collects shopper credentials and exchanges them with Cognito for authentication tokens. These tokens are subsequently attached to requests made to the protected /cart resource in API Gateway, ensuring that only authenticated shoppers can view or modify a cart.

- User Pool configured for self-service sign-up and sign-in
- Tokens issued on successful authentication and used to authorize API calls
- Decouples identity management from the application's business logic

5.3. REST API Configuration with Amazon API Gateway

The SeasonalShoppingAPI REST API was created in API Gateway with resources for /products and /cart. The /products resource allows the storefront to retrieve catalogue information, while

the /cart resource supports the operations needed to add, update, remove, and view items in a shopper's cart. A Cognito authorizer was attached to the /cart resource so that only requests carrying a valid token are forwarded to the backend.

- /products and /cart resources exposed as a versioned REST API
- Cognito authorizer protecting cart-related operations
- Lambda proxy integration forwarding requests to ManageShoppingCart

5.4. AWS Lambda for Cart and Product Processing

The ManageShoppingCart Lambda function, written in Node.js/Python, was developed to handle the business logic behind both API resources. For /products requests, it returns catalogue data used to render the storefront; for /cart requests, it validates the payload and performs the corresponding create, read, update, or delete operation against DynamoDB before returning the updated cart to the caller.

Key Lambda details:

- Runtime: Node.js / Python
- IAM execution role scoped to DynamoDB read/write and CloudWatch Logs
- Stateless design — each invocation is independent and short-lived

5.5. Data Storage with Amazon DynamoDB

A DynamoDB table, ShoppingCarts, was created to store cart data.

- Table name: ShoppingCarts
- Primary key: cartId (string)
- Item attributes: product entries, quantities, and cart metadata

Features:

- Automatic scaling to handle varying shopper traffic
- Real-time, low-latency data access for reads and writes
- Can be extended with global secondary indexes for order history or analytics (future scope)

5.6. IAM Roles and Security

All interactions between services are secured using IAM roles configured with the principle of least privilege.

- Lambda execution role: read/write access to the ShoppingCarts table and permission to write CloudWatch Logs
- API Gateway: restricted to invoke only the ManageShoppingCart function
- Cognito: issues and validates tokens used to authorize protected API calls

5.7. Infrastructure Deployment with AWS CloudFormation

The complete set of resources — the S3 bucket, the Cognito User Pool, the API Gateway API and its resources, the Lambda function, the DynamoDB table, and the IAM roles — was captured in an AWS CloudFormation template and deployed as a single stack. This ensured that every resource was created with consistent configuration and that the deployment could be repeated or updated without manual console changes.

- Single CloudFormation stack describing the entire architecture
- Consistent, repeatable deployment across environments
- Stack outputs expose the website endpoint and API URL for consumption

5.8. Sample API Interaction

The table below illustrates a representative request and response pair for the /cart resource, showing how a shopper's browser adds a product to their cart through the SeasonalShoppingAPI.

Step	Detail
Request	POST /cart with header Authorization: <Cognito ID token> and JSON body { "cartId": "c-1042", "productId": "P-118", "quantity": 2 }
API Gateway	Validates the token with the Cognito authorizer, then invokes ManageShoppingCart via a Lambda proxy integration

Step	Detail
Lambda	Validates the payload, looks up cartId c-1042 in ShoppingCarts, and appends or updates the productId item
DynamoDB	Persists the updated cart item under the cartId partition key
Response	HTTP 200 with the updated cart JSON returned to the storefront for display

5.9. Error Handling and Validation

The ManageShoppingCart Lambda function was designed to fail safely and to return informative errors rather than allow malformed data to reach DynamoDB. Incoming payloads are validated for required fields such as cartId and productId before any database call is attempted; requests missing these fields return an HTTP 400 response. Requests without a valid Cognito token are rejected by API Gateway's authorizer before they ever reach Lambda, returning an HTTP 401 response. Any unexpected failure inside the function — for example, a transient DynamoDB throttling event — is caught, logged to Amazon CloudWatch Logs for diagnosis, and surfaced to the caller as an HTTP 500 response rather than as a silent failure.

This layered validation approach — authorization at the API Gateway layer, payload validation at the Lambda layer, and structured error responses at every stage — keeps the ShoppingCarts table free of malformed records and gives the storefront enough information to show a meaningful message to the shopper when something goes wrong.

The successful implementation of this serverless system showcases how AWS managed services can be integrated to create a real-time, low-maintenance, and scalable e-commerce storefront. The modular, event-driven approach ensures flexibility for future enhancements such as payment integration, content delivery acceleration, or a recommendation engine.

6. RESULTS

The implementation of Seasonal Styles Hub yielded several successful outcomes that align with the project's objectives. The system was tested under multiple scenarios to validate functionality, performance, scalability, and security. The following results highlight the effectiveness of using a serverless architecture on AWS for a seasonal e-commerce storefront.

6.1. Reliable Static Storefront Delivery

The seasonal-styles-hub-v2 S3 bucket consistently served the Home, About Us, Contact, Cart, and Sign In pages over HTTPS with fast response times, confirming that Amazon S3 is well suited to hosting a static storefront without dedicated web servers.

6.2. Secure Shopper Authentication

The Cognito User Pool successfully registered and authenticated shoppers, issuing tokens that were correctly validated by the Cognito authorizer attached to the /cart resource in API Gateway. Requests without a valid token were rejected, confirming that cart operations remain protected from unauthenticated access.

6.3. Responsive Cart Operations via the REST API

The SeasonalShoppingAPI processed /products and /cart requests through the ManageShoppingCart Lambda function with low, predictable latency. Adding, updating, and removing items from a cart correctly updated the corresponding record in the ShoppingCarts DynamoDB table, confirming that the API, Lambda, and database layers work together as designed.

6.4. Scalable, Serverless Architecture

The system handled varying volumes of storefront traffic and cart requests without any manual scaling or downtime, thanks to the underlying serverless AWS infrastructure. The combination of S3, Cognito, API Gateway, Lambda, and DynamoDB ensures that each component functions independently and scales automatically based on demand.

Testing scenarios included:

- Browsing the storefront and seasonal category pages
- Signing up and signing in as a new shopper
- Adding, updating, and removing items from the cart in quick succession

All scenarios were handled successfully without performance degradation.

6.5. Secure and Controlled Service Access

Using AWS IAM roles and policies, secure access was configured between all services.

- The Lambda function could only access the ShoppingCarts table and CloudWatch Logs
- API Gateway resources were restricted to invoke only the intended Lambda function
- Cognito-issued tokens were required for all protected cart operations

This validated that the system follows best practices for cloud security and least-privilege permissions.

6.6. Repeatable Deployment and User Experience

The AWS CloudFormation template consistently recreated the full environment, confirming the value of an infrastructure-as-code approach for this project. From a shopper's perspective, the platform requires minimal interaction: browse the seasonal catalogue, sign in, and manage the cart, with every subsequent action handled automatically by the underlying serverless services.

6.7. Performance Observations

Across repeated test runs, the storefront's static pages loaded quickly from the S3 website endpoint, and the SeasonalShoppingAPI responded to /products and /cart requests with consistent, low latency, reflecting the single-digit-millisecond read and write performance of DynamoDB combined with the short, stateless execution time of the Lambda function. No degradation in response time was observed as the number of simulated concurrent shoppers increased, which is consistent with the automatic scaling behaviour built into API Gateway, Lambda, and DynamoDB.

6.8. Summary of Outcomes

The table below summarises the outcome observed for each objective set out at the start of the project.

Objective	Outcome
Global storefront availability	Achieved via S3 static website hosting over HTTPS
Secure shopper accounts	Achieved via Cognito User Pool registration and sign-in
Reliable cart operations	Achieved via API Gateway, Lambda, and DynamoDB
Least-privilege security	Achieved via scoped IAM roles for Lambda
Repeatable deployment	Achieved via a single AWS CloudFormation stack

The overall results demonstrate that the Seasonal Styles Hub architecture meets its intended goals of automation, security, scalability, and ease of use. The architecture is stable and ready for real-world operation, with clear opportunities for further expansion and integration.

7.CONCLUSION

The successful design and deployment of Seasonal Styles Hub demonstrates the practicality of cloud-native, serverless architectures in solving real-world e-commerce challenges. By leveraging key AWS services such as S3, Cognito, API Gateway, Lambda, DynamoDB, IAM, and CloudFormation, the system achieves its core objectives: hosting a global, always-available storefront, authenticating shoppers securely, and processing shopping-cart operations reliably — all without managing any physical or virtual infrastructure.

The project highlights the benefits of serverless computing, including automatic scaling, cost-efficiency, fault tolerance, and reduced operational complexity. Amazon S3 delivers the storefront at global scale with no server management, while Amazon Cognito provides a secure, managed identity layer that would otherwise require significant custom development. The pairing of API Gateway and Lambda gives the platform a flexible, on-demand backend, and DynamoDB provides fast, reliable storage for cart data. AWS CloudFormation ties the whole architecture together as a single, reproducible infrastructure-as-code deployment.

Throughout the development lifecycle, careful attention was given to security, performance, and modularity. The system was designed using least-privilege IAM permissions, token-based authorization through Cognito, and a decoupled, event-driven backend. Testing confirmed the system's ability to serve the storefront, authenticate shoppers, and process cart operations reliably under a range of conditions.

Beyond its current functionality, the project lays a strong foundation for future enhancements, such as:

- Integrating a payment gateway to support end-to-end checkout
- Placing Amazon CloudFront in front of the S3 storefront for global content delivery acceleration
- Adding order-history tracking and a customer account dashboard
- Introducing personalized product recommendations, for example using Amazon Personalize
- Exposing analytics dashboards over cart and catalogue activity

In conclusion, this project serves as a practical and scalable blueprint for serverless e-commerce application development, particularly in domains where global reach, security, and real-time responsiveness are critical. It showcases how AWS managed services can be strategically combined to build an efficient, secure, and future-ready shopping platform with minimal operational burden.